

Lab Exercise 1 – From Manual Threads to Executor Framework

Lab Title: Building an E-Commerce Order Processing System Using Threads and ExecutorService

Objective

In this lab, you will build the same application twice to understand the difference between manually managing threads and using the modern **Executor Framework**.

Phase 1

Create and manage threads manually using `new Thread().start()`.

Phase 2

Refactor the same application using the `ExecutorService`.

By the end of this lab, you will be able to:

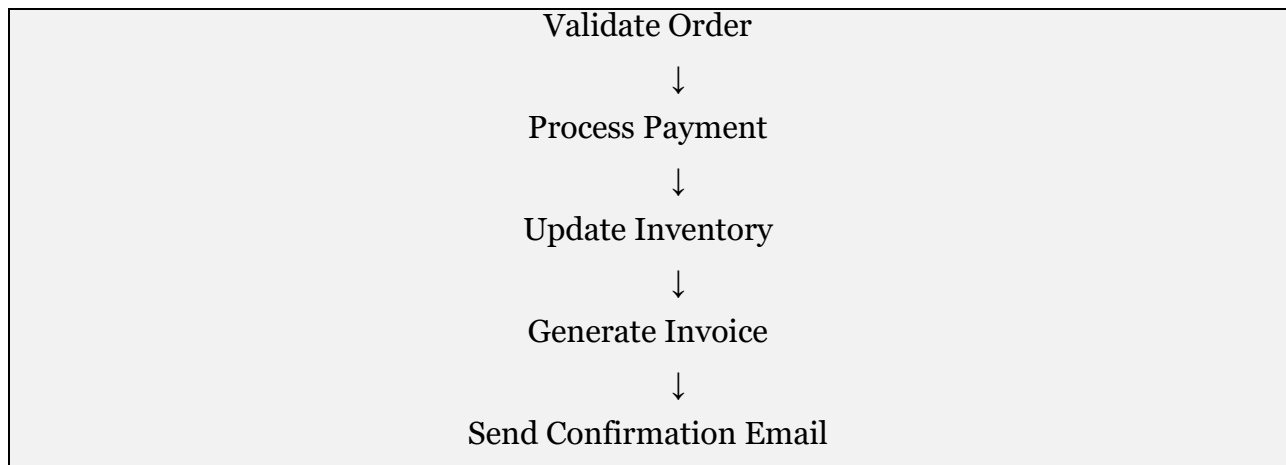
- Understand the drawbacks of creating threads manually.
- Learn how the Executor Framework manages tasks efficiently.
- Observe thread reuse using a thread pool.
- Compare both approaches in terms of performance, scalability, and code maintainability.

Scenario

You are working as a **Java Developer** for an **E-Commerce Company**.

Whenever a customer places an order, the system performs multiple independent operations before the order is completed.

Each customer order requires the following five tasks:



The company expects to process **10 customer orders simultaneously**.

Each task takes approximately **2 seconds** to complete.

Business Requirement

For every customer order, the following tasks should execute independently.

Task	Description
Validate Order	Verify customer and product details

Task	Description
Process Payment	Deduct payment from customer's account
Update Inventory	Reduce product quantity from stock
Generate Invoice	Create the customer's invoice
Send Confirmation Email	Send order confirmation email to the customer

Lab Requirements

You will implement the application in **two different ways**.

Phase 1 – Manual Thread Management

Implement the application using **new Thread().start()**.

For each customer order:

- Create **5 separate threads**
- Each thread performs one business task
- Start all five threads
- Process **10 customer orders**

Expected Thread Creation

Customer 1

Thread-1 → Validate Order

Thread-2 → Process Payment

Thread-3 → Update Inventory

Thread-4 → Generate Invoice

Thread-5 → Send Confirmation Email

Customer 2

Thread-6 → Validate Order

Thread-7 → Process Payment

Thread-8 → Update Inventory

Thread-9 → Generate Invoice

Thread-10 → Send Confirmation Email

Continue this process until **Customer 10**.

Phase 1 - Complete Solution

File Name

ManualOrderProcessing.java

```
public class ManualOrderProcessing {

    public static void main(String[] args) {

        System.out.println("=====
");
        System.out.println(" E-Commerce Order Processing");
        System.out.println(" Using Manual Threads");

        System.out.println("=====
");

        for (int customer = 1; customer <= 10; customer++) {

            int customerId = customer;

            Thread validateThread =
                new Thread() -> performTask(customerId, "Validate Order"));

            Thread paymentThread =
                new Thread() -> performTask(customerId, "Process Payment"));

            Thread inventoryThread =
                new Thread() -> performTask(customerId, "Update Inventory"));

            Thread invoiceThread =
                new Thread() -> performTask(customerId, "Generate Invoice"));

            Thread emailThread =
```

```

        new Thread() -> performTask(customerId, "Send Confirmation Email"));

        validateThread.start();
        paymentThread.start();
        inventoryThread.start();
        invoiceThread.start();
        emailThread.start();
    }

}

public static void performTask(int customerId, String taskName) {

    try {

        System.out.println("-----");
        System.out.println("Customer ID : " + customerId);
        System.out.println("Task      : " + taskName);
        System.out.println("Thread Name : "
            + Thread.currentThread().getName());

        Thread.sleep(2000);

        System.out.println(taskName + " Completed");
        System.out.println("-----");

    } catch (InterruptedException e) {

        e.printStackTrace();

    }

}

}

```

Observe the Output

Students should notice thread names such as:

Thread-0

Thread-1

Thread-2

Thread-3

Thread-4

...

Thread-49

Discussion

- Total Customers = **10**
- Tasks per Customer = **5**
- Total Tasks = **50**
- Total Threads Created = **50**

Every task creates a **new thread**.

Phase 2 – Refactoring Using ExecutorService

The development team decides to improve the application.

Instead of creating a new thread for every task, the company creates a **Thread Pool** containing **5 worker threads**.

Every task will be submitted to the Executor.

The Executor decides which thread executes the task.

Phase 2 - Complete Solution

File Name

ExecutorOrderProcessing.java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorOrderProcessing {

    public static void main(String[] args) {

        System.out.println("=====
");
        System.out.println(" E-Commerce Order Processing");
        System.out.println(" Using Executor Framework");

        System.out.println("=====
");

        ExecutorService executor =
```



```

        Executors.newFixedThreadPool(5);

for (int customer = 1; customer <= 10; customer++) {

    int customerId = customer;

    executor.execute(() ->
        performTask(customerId, "Validate Order"));

    executor.execute(() ->
        performTask(customerId, "Process Payment"));

    executor.execute(() ->
        performTask(customerId, "Update Inventory"));

    executor.execute(() ->
        performTask(customerId, "Generate Invoice"));

    executor.execute(() ->
        performTask(customerId, "Send Confirmation Email"));

}

executor.shutdown();

}

public static void performTask(int customerId, String taskName) {

    try {

        System.out.println("-----");
        System.out.println("Customer ID : " + customerId);
        System.out.println("Task      : " + taskName);
        System.out.println("Thread Name : "
            + Thread.currentThread().getName());
    }
}

```

```
        Thread.sleep(2000);

        System.out.println(taskName + " Completed");
        System.out.println("-----");

    } catch (InterruptedException e) {

        e.printStackTrace();

    }

}

}
```

Observe the Output

Students should notice only these thread names:

```
pool-1-thread-1
pool-1-thread-2
pool-1-thread-3
pool-1-thread-4
pool-1-thread-5
```

Even though there are **50 tasks**, only **5 worker threads** execute them.

The threads are **reused**.

Lab Comparison

Feature	Manual Threads	Executor Framework
Customers	10	10
Tasks per Customer	5	5
Total Tasks	50	50
Threads Created	50	5
Thread Reuse	No	Yes
Task Queue	No	Yes
Memory Usage	High	Low
Performance	Lower	Better
Scalability	Poor	Excellent
Enterprise Ready	No	Yes

Challenge Activities

Challenge 1

Increase the number of customers from **10** to **100**.

Observe:

- How many threads are created in the manual solution?
 - How many threads are created in the Executor solution?
-

Challenge 2

Increase the task delay from:

```
Thread.sleep(2000);
```

to

```
Thread.sleep(5000);
```

Observe:

- Which solution performs better?
 - Which solution is easier to manage?
-

Challenge 3

Modify the thread pool size.

Change:

```
Executors.newFixedThreadPool(5);
```

to

```
Executors.newFixedThreadPool(2);
```

Observe:

- Which tasks wait in the queue?
- Which thread executes the most tasks?

Challenge 4

Replace:

```
Executors.newFixedThreadPool(5);
```

with

```
Executors.newSingleThreadExecutor();
```

Observe:

- How are the tasks executed?
- Are they processed one after another?

Reflection Questions

1. How many threads were created in the manual solution?
2. How many threads were created in the Executor Framework solution?
3. Why are the thread names repeated in the Executor Framework output?
4. What happens if the application receives **1,000 customer orders**?
5. Which approach would you recommend for a banking or e-commerce application, and why?
6. What is the primary advantage of separating **tasks** from **threads**?

7. Why is `ExecutorService` considered the modern standard for concurrency in Java?
-

Expected Learning Outcome

After completing this lab, students will be able to:

- Create and execute threads manually using `new Thread().start()`.
- Identify the limitations of manual thread management.
- Refactor a multithreaded application using the `ExecutorService`.
- Understand how a **Fixed Thread Pool** reuses worker threads.
- Compare manual threading with the Executor Framework based on scalability, memory usage, and performance.
- Appreciate why modern Java frameworks such as **Spring Boot**, web servers, and enterprise applications use the **Executor Framework** instead of creating threads manually.